Derived Plans, Derived Crashes:

The Synthesis Substrate, Third Edition —
Authorship Replaced by Derivation, for Success *and* for Failure
saturate → sequence → execute → verify, now under derived supervision: topology → geometry → named branches → lawful recovery

Sean Chatman
with the Praxis working system, `crates/praxis-synthesis` v26.7.2

2–3 July 2026 — Genesis, Days 9–10 (third edition: the fault-tolerant plans)

Abstract

The first edition of this paper showed a plan that had been hand-authored becoming a *derived* artifact: from declared capabilities and a saturated fact base, a bounded solver discovers order and bindings, executes the result as a content-addressed DAG, and admits it through refinements that recompute rather than trust. The second edition traced every layer's scientific supply chain. Both editions shared a hidden assumption with almost all planning literature: **every step succeeds**. This edition removes it. Erlang's answer to failure — isolate, supervise, restart — was designed for a world where the crash space could not be seen in advance; the instruments of modern physics (the LHC's triggers, the EHT's reconstruction) teach the opposite discipline: predict the event signatures, then measure. We fuse the two: the *planner's own analysis* now derives, before runtime, a supervision topology (stages and restart strategies earned from the plan's data-dependency structure) and a failure geometry (per-node ordered branches: named class, signal conjunction, lawful response), both content-addressed into the plan's receipt lineage. At runtime, crashes are values; every crash lands in a named branch or in the implicit, unshadowable GeometryGap; the four lawful responses (restart within an 8-bounded intensity, park with a guaranteed way back, refuse with a certificate naming the culprit fact, escalate) actuate under receipt; surrender itself is an *Ok* result. The machinery is measured, not asserted: at 10,000 supervised cell members, supervision overhead is statistical noise, throughput is flat across fault rates from 0% to 50%, recovery counts track injected rates exactly, and a crash-looping workload class is quarantined by a cross-group quorum within one epoch — while an unmodified foreign verifier still accepts every receipt. The design is a distillation of a real lineage (CNS's 8T/8H/8M doctrine, BitActor's tick budgets, ByteActor's bounded rings, knhk's classification-without-actuation), ported with receipted provenance and zero new dependencies. The thesis, extended: **authorship placed correctness in a head; derivation places it in a chain — and a chain that names its own failure modes before they occur is the only kind a trillion agents can share.**

Contents

1 The Removed Assumption

A derived plan is a proof object: the solver's search establishes that the step sequence reaches the goal *if each step behaves*. The first two editions stopped there, as does most of the planning literature — PDDL semantics, like Hoare triples, speak of what holds *after* an action, never of the action dying halfway. Real fleets die halfway constantly. The question this edition answers:

> *Can failure handling be derived from the same declarations the plan was derived from — so that recovery, like the plan, is a proof object rather than an improvisation?*

Two traditions supply half-answers each.

Lineage 1.1 (Armstrong: failure as a normal, supervised event). Erlang/OTP [1] made three moves that survive every re-examination: processes are isolated so failure cannot spread by memory; supervisors — not the failing code — own the recovery decision; and restart strategies (`one_for_one`, `rest_for_one`, `one_for_all`) plus a *restart intensity* (max N restarts per window, then give up upward) turn "let it crash" into a bounded, structured discipline. What OTP does *not* do is derive the tree: a human writes the supervision hierarchy, and the crash space — what can fail, with what signature, deserving what response — lives in the programmer's head. Armstrong designed for a world where the far side of the moon could not be seen.

Lineage 1.2 (The instrument sciences: predict, then measure). The LHC does not record every collision; its trigger system encodes, in advance, the signatures of events worth keeping [2]. The EHT did not photograph a black hole; it reconstructed one from lawful projections gathered by an instrument *designed around a predicted signal* [3]. The discipline common to both: enumerate the event geometry before the event, instrument for exactly those signatures, and treat anything outside the geometry as a first-class finding — never as noise to be discarded silently.

Lineage 1.3 (The 8-bounded actor lineage: CNS → BitActor → ByteActor → knhk). A parallel, in-house lineage spent 2025–2026 building the execution-side half. CNS fixed the doctrine: **8T/8H/8M** — at most 8 ticks per hot operation, 8 hops per workflow, 8-byte quantization — and shipped the canonical BitActor engine (`BITACTOR_TICK_BUDGET 8`, zero-tick fast path, BLAKE3-sealed spec↔exec) [4]. ByteActor V3 productized it: 64-byte crystal envelopes, per-core bounded SPSC rings (the explicit anti-Akka move: no unbounded

mailboxes, no GC pauses), "Erlang coordinates, C executes" [5]. knhk carried the doctrine into ∼250K lines of Rust: a branchless `TickBudget` with the Chatman constant, a compile-time `ChatmanBounded` assertion, an R1/W1/C1 runtime-class taxonomy with per-class failure policies, and a `ParkManager` that demotes over-budget work with a typed cause [6]. The lineage's three invariants never wavered: declared semantics compiled down; 8-bounded execution with a zero-cost fast path; hash-sealed provenance. And the lineage's one persistent gap never closed: **classification without actuation**. knhk can say *what kind* of failure occurred and *what should* happen; nothing in the constellation restarts, re-admits, or closes the loop. Its parked work has no way back; its MAPE-K loop re-plans overlays but restarts nothing.

This edition is the fusion: OTP's actuation vocabulary, the instrument sciences' predict-then-measure stance, the lineage's bounded machinery — and the synthesis substrate's own novelty, *derivation*, applied to all of it. (Dependency note, receipted: the knhk machinery is *ported*, not linked — live build probes showed even its leanest crate drags an async/telemetry stack, and its receipt algebra is SHA3/ed25519 against praxis's BLAKE3 trust root. Every port carries a greppable `PORT(knhk)` provenance tag; the crate gained zero new dependencies.)

2 Derived Supervision Topology

Definition 2.1 (Dependency depth and stages)**.** Let $G = (V, E)$ be the plan's data-dependency DAG (edge $u \to v$ iff an add-effect of u feeds a precondition of v; first edition, §4). Define $d(v) = 0$ for sources and $d(v) = 1 + \max_{u \to v} d(u)$ otherwise. A *stage* $S_k = \{v : d(v) = k\}$.

Definition 2.2 (Earned strategy)**.** Stage S_k supervises RestForOne iff some $v \in S_k$ has a nonempty transitive dependent set $D(v)$; otherwise OneForOne. The *cohort* of v is $\{v\} \cup D(v)$ under RestForOne and $\{v\}$ under OneForOne.

The strategy is not chosen; it is *earned* by position: a producer whose outputs feed later work restarts together with that work, because its failure invalidates them; a leaf restarts alone. Two OTP strategies are deliberately absent, and the absences are load-bearing:

Refusal 2.3 (No OneForAll). OTP justifies one-for-all by shared mutable fate between siblings. In an acyclic data-flow plan, same-depth siblings are independent *by construction* — the coupling one-for-all encodes is inexpressible. The variant does not appear in the `Strategy` enum at all; an exhaustive-match test makes its future addition a compilation event, forcing the doctrine conversation.

Refusal 2.4 (No SimpleOneForOne; intensity > 8 refused, not clamped). Derived plans have static step sets (dynamic children arrive via the incremental-assertion path, edition one §2.4). And `RestartPolicy::new(9, _)` returns a Refusal — the byte governor applies to recovery itself, and silently clamping a stated policy would be the executor editing the operator's law.

Proposition 2.5 (The topology is a value)**.** $\mathcal{T} = (\text{stages}, \text{policy})$ *is produced only by* derive *(a sealed constructor: no literal construction compiles), and* topology_hash $= \mathsf{ca}(\text{stages} \parallel \text{policy} \parallel \text{plan\_hash} \parallel \text{proble}$ *joins the receipt lineage. Determinism is test-pinned: same plan, same hash.*

On the five-step lawobject chain the derivation yields five singleton stages, the first four RestForOne (each feeds everything after it) and the leaf OneForOne; the root's cohort is the whole plan, the leaf's is itself — exactly the tree an Erlang engineer would have written by hand, now emitted by the same machinery that emitted the plan.

3 Derived Failure Geometry

3.1 Classes, signals, responses

Definition 3.1 (The eight classes)**.** FailureClass = {LogicFault, BudgetBreach, AuthorityVacuum, TransientFault, Stall, StarvedInput, CertifiedUnsat, GeometryGap} — the semantic axis, capped at eight by the doctrine. knhk's R1/W1/C1 tiers ride orthogonally as data on each crash snapshot; fusing the axes would yield 24 variants, which is how taxonomies die.

Definition 3.2 (Branch, geometry)**.** A *branch* is (c, Σ, r): a class, a conjunction of signal predicates over a crash snapshot (retries-at-least, ticks-above-budget, crash-kind, refusal-head, no-progress, upstream-parked), and a lawful response $r \in \{\mathsf{Restart}, \mathsf{Park}(\rho), \mathsf{Refuse}(\text{core}), \mathsf{Escalate}\}$. The *geometry* Γ maps each node to an ordered branch list; classification is first-match-wins; geometry_hash = $\mathsf{ca}(\Gamma \parallel \text{topology\_hash})$.

Theorem 3.3 (The gap is unshadowable)**.** *GeometryGap is the value classification returns when no stored branch matches; it is never a member of any stored list (a structural invariant, test-enforced over every derived geometry). Consequently no branch ordering, addition, or deletion can capture, mask, or remove the gap outcome: the map admits its own incompleteness* by construction*, not by convention.*

Proof. Immediate: an unstorable value cannot be shadowed by stored values, and first-match-wins terminates in the implicit case exactly when all stored matches fail. The enforcement is the derivation function's type: branch lists are built from a constructor set that excludes the gap class. □

3.2 Mining the branches from the plan's own analysis

The geometry is not a template; parts of it are *computed from the problem.* The decisive case:

Definition 3.4 (Fragile fact)**.** A precondition predicate p of capability c is *fragile* iff **no** capability in the problem produces p. The initial state is a one-time gift: if a fragile fact is lost mid-run, nothing in the plan can lawfully re-produce it — whereas a fact with even one producer is recoverable by restarting that producer.

For every fragile fact, derivation emits an AuthorityVacuum branch responding Refuse([MissingFact(p)]): the same producer analysis that powers Solver8's pre-search unsat certificates (edition one, §3), now aimed at runtime loss. In the self-application domain, `push`'s `authorization` precondition is fragile; the derived geometry therefore guards, *before runtime*, exactly the gate the release doctrine cares about — and a runtime revocation of authorization refuses in **one crash, zero futile retries**, with the fact named in the certificate (test-pinned).

An instructive honesty note: the first implementation of fragility ($\neg\text{init} \wedge \text{producers} \leq 1$) was wrong in both directions — it guarded recoverable facts and missed init-only ones — and was caught by its own two tests failing with *inverted* expectations. The corrected rule (producers = 0) is the one proved above. The method continues to catch its author.

The remaining branches encode doctrine as order: a tick breach (TicksAboveBudget) refuses *before* the time-tier over-budget branch can park it — the hot path never retries over budget (knhk's R1 discipline, upgraded from a metrics counter to a certificate); Stall (second attempt, no progress) outranks TransientFault; a bad output earns exactly one restart and then a Manual park, because *retry does not heal wrong computation.*

4 Supervised Execution: the Loop Closed

4.1 Crashes are values

Under `forbid(unsafe_code)` there are no panics to catch. The executor's runner trait returns Result⟨(bytes, ticks), NodeCrash⟩, with a blanket implementation over every infallible runner — existing code is untouched, and a crash is data: Io, BadOutput, OverBudget, PreconditionLost, Refused. This is the lineage's crystal-envelope instinct (state that travels as a small, total value) applied to failure itself.

4.2 Actuation semantics

On crash: snapshot → classify → named branch → actuate, every step receipted into a chain seeded fold(H(`geometry/v1`), geometry_hash) — so the chain's genesis *is* the derived map, and replaying it proves recovery followed the geometry rather than improvisation.

- **Restart**: retry within intensity; upstream work replays free through the memo/WAL layer (edition one's kill -9 durability composes here unchanged — process death mid-restart-loop recovers to the same receipt).
- **Park**: a ParkedEntry with a derived re-admission policy — OnInputChange (something upstream was fixed), AfterRuns(n), or Manual (only authority re-admits) — closing knhk's no-way-back gap; parked entries persist through the same BLAKE3 WAL, so *quarantine survives machine death.* Dependents are skipped with attribution (SkippedBy), never silently.
- **Refuse / Escalate**: the run halts with the branch's certificate inside the receipt; the crash chain is never lost to the halt.
- **Surrender**: intensity exhaustion yields RunOutcome::GaveUp — an *Ok*, not an *Err*. Lawful surrender is a result with standing; only lawless states are errors.
- **The gap**: an unmatched crash emits a GeometryGap receipt, takes the safe default (restart), and the run *completes* — with geometry_conformance = false reported honestly. A gap is a finding, not a failure.

Proposition 4.1 (Total accounting)**.** *Every node of every supervised run carries exactly one disposition (Completed(r), Parked, SkippedBy, GaveUp); no silent rows exist (test-pinned:* $|\text{dispositions}| = |V|$*).*

Measurement 4.2 (Recovery is invisible in the artifact). On the lawobject plan with two injected transient crashes at `judge`: both crashes land in named branches (TransientFault, then Stall — the second identical crash correctly classified as no-progress), the run completes, and the final root hash is **byte-identical** to the crash-free run's. Park-then-re-admit heals to the same identity: run 0 parks the over-budget node and skips its dependents; run 1's re-admission pass completes the plan with upstream served from memo, root again byte-equal to crash-free. Machine death composes: kill -9 mid-restart-loop, WAL recovery, identical receipt.

5 The Supervised Cell: Composition Under Failure

Supervision must not break the cell's composition law (the cell hash is a function of group roll-up roots alone; edition one's minimum phase-change cell). Three design rules keep it intact, each test-enforced:

1. **Additive-only receipts.** New counters (recovered, parked, geometry_gaps, restarts, the quarantine set, the epoch-plan chain) are serde-defaulted fields *outside* all hashed material. The invariant admitted + refused = n holds (parks live inside refused; recoveries inside admitted), the fold functions are byte-identical — and the **unmodified** Python foreign verifier accepts every supervised receipt (tested).

2. **Combo status lanes.** All eight status-byte bits are taken, so supervision claims provably-unreachable *combinations*: S_RECOVERED = $H|A$ (the base run never co-sets halted with admitted — proven by enumeration), S_PARKED = $H|B$, S_GEOMETRY_GAP = $H|U|E$. agent8's SWAR sweep is compatible by construction: a recovered member still carries A and still counts as admitted at 9.76 G agents/s with zero code change.

3. **Determinism through faults.** Injection is a seed-keyed splitmix64 lottery; a member's byte, restarts, and terminal hash replay exactly given the seed (pinned over 24 agents × 2 runs), so selective challenge survives supervision.

5.1 MAPE-K, closed at epoch boundaries

knhk's MAPE-K loop analyzed and never actuated. Here it closes, with one discipline: *parameters change only at epoch boundaries* — mid-epoch mutation is refused by construction (the quarantine set is immutable within an epoch), preserving per-epoch replay. The roll-ups *are* the monitor plane; analysis detects a crash-looping template by **cross-group quorum** (the same template parking in ≥ 3 distinct groups in one epoch — replication across independent shards is the evidence bar that the template, not the host, is at fault); the plan is a receipted EpochPlan whose hash chains under its own domain; the execute step is the next epoch's quarantine set, whose refusals surface in the *existing* refusal register through the *existing* bucketing — zero register changes.

Measurement 5.1 (The supervised cell, 10,000 members, release). Overhead of supervision at fault rate 0: -0.9% *of medians over five paired runs*, inside the baseline's own $\pm 5.7\%$ run spread — statistical noise; the healthy path is untaxed. (The first edition of this measurement reported -2.7% from a *single* sample; the harness now carries a self-refutation guard — a negative overhead exceeding the baseline spread *fails* the run instead of being published — and the single-sample figure is retired.) Per-member latency at 10% faults: p50 34 μs, p99 1.42 ms, worst 2.29 ms — the tail the aggregate throughput was masking; verdicts are percentiles and worst cases, never minimums. Composition ratio (whole cell over the sum of individually timed members): 1.0004 — the roll-up machinery's priced cost. Recovery counts track injected transient rates exactly: 69/608/3,179 recovered at 1%/10%/50%. Throughput is **flat** across all fault rates ($\sim$15,000 members/s, recomputed from count and elapsed time). The crash-looping template is detected at quorum and quarantined by epoch 2 in every configuration; final-epoch members of that class refuse cheaply instead of burning restarts. Every cell verifies in Rust, and `foreign_verify.py` — byte-unmodified — verifies the supervised receipts. (Receipt: `receipts/supervised_cell.json`; regenerable by the committed harness.)

Remark 5.2 (What flat throughput means). The cost of a supervised fleet tracked its *failure novelty*, not its failure *rate*: recovered transients are absorbed at boundary cost, and the one genuinely novel pathology (the crash-looping class) was converted — once, by quorum — into a cheap standing refusal. This is the overlap-curve law of edition one (*deliberation cost is linear in novelty, invariant in fleet size*) reappearing on the failure side, which is precisely what the trillion-agent thesis predicts supervision must do. The full novelty-curve-under-faults re-measurement is receipted as deferred (Refusal 7.3), not claimed.

5.2 The Rust Fable Testbed: Spec-Driven LLM Evaluation

The `rust-fable-testbed` crate applies the synthesis substrate's oracle discipline to the evaluation of LLM-generated code. Instead of trusting that a model's output is correct, every proposed code block is subjected to a multi-stage compilation and test oracle before being admitted into the plan's receipt lineage.

Definition 5.3 (Fable harness and mock model membrane)**.** A *Fable harness* is a test environment that:

(1) constructs a *prompt* P_T from a task ontology T (a typed description of the coding task, its inputs, outputs, and invariants),

(2) calls a *mock model membrane* M that intercepts the LLM API, returning a deterministic response drawn from a test fixture indexed by P_T's hash,

(3) extracts the code block C from the model response R, and

(4) runs the verification oracle $\mathrm{Oracle}(T, R)$ (Definition 5.7) against C in an isolated `cargo` workspace.

The mock membrane ensures that harness runs are hermetic: no real LLM call is made, the response is deterministic given P_T, and the oracle verdict is reproducible.

Proposition 5.4 (The Fable oracle is a decidable oracle in the sense of Def. 4.1 of Paper 04)**.** $\mathrm{Oracle}(T, R)$ *as defined in Definition 5.7 is a decidable oracle for the correctness question "does C implement T correctly?" on the finite test corpus generated by the harness: it is total, terminating, and labelling (pass or fail with a structured error from the failing stage).*

Proof. Each stage of the oracle (`cargo build`, `cargo test`, `cargo clippy`, `safety_audit`) terminates on every finite C (the Rust compiler and test runner always terminate on finite input with no infinite loops in the toolchain). The codomain of each stage is $\{0, 1\} \times \mathrm{ErrInfo}$, a total finite type. A failure at any stage short-circuits with the stage index and its `ErrInfo`, analogous to the staged validator of Paper 04's Definition 4.7. Hence Oracle is total and terminating, and by Proposition 4.4 of Paper 04 any observed failure is a genuine defect in C, not a property of the oracle. □

Construction 5.5 (Receipt chaining for LLM code artifacts)**.** When $\mathrm{Oracle}(T, R) = 1$, the harness mints a receipt using the standard `OcelCausalFrame` chain:

$$h_+ \;=\; \mathsf{H}\big(h_- \,\|\, \mathrm{body}(\mathsf{fr})\big), \tag{1}$$

where the frame's `obj_refs` carry $\mathrm{dg}(C) = \mathsf{H}(\text{canonical bytes of } C)$ (Construction 6.1 for the group-level context and Definition 6.2 for WAL durability). The genesis seed is $\mathsf{H}(\text{"fable-v1-genesis"})$, domain- isolated from the plan chain so fable receipts cannot cross-verify with plan receipts even if their terminal hashes collide.

Remark 5.6 (Mock membranes as differential oracles)*.* The mock membrane produces responses from test fixtures; a real-model run produces a different response. Running both and comparing (differential oracle, Proposition 4.2 of Paper 04) bounds the false-accept rate: an implementation error that passes the oracle with the mock response but fails with the real response is caught by the differential. This is the `rust-fable-testbed`'s production validation path: mock fixtures provide determinism; occasional real-model probes bound the divergence.

5.3 Spec-Driven Sandbox Verification Oracles

LLM evaluation harnesses evaluate code correctness using mock model membranes and sandboxed execution environments.

Definition 5.7 (Sandbox Verification Oracle)**.** Let P_T be a prompt compiled from task ontology T, and let C be the code block extracted from model response R. The sandbox verification oracle is the composite:

$$\text{Oracle}(T, R) \;=\; \text{cargo\_build}(C) \wedge \text{cargo\_test}(C) \wedge \text{cargo\_clippy}(C) \wedge \text{safety\_audit}(C). \quad (2)$$

The outcome is receipted and chained to the ledger using rolling BLAKE3 hashes.

6 Hierarchical Merkle Cell Rollups and Write-Ahead Logs

To scale verification, agents are grouped into hierarchical Merkle rollups.

Construction 6.1 (Hierarchical Merkle Cell Rollups)**.** Let $M_{g,i}$ be the terminal hash of member i in group g. The Group Receipt hash is:

$$H_g \;=\; \mathsf{H}\Big(\,\text{sort}\big(\{M_{g,i}\}\big)\,\Big), \quad (3)$$

and the Cell Receipt hash is the rolling fold over G group hashes:

$$H_{\text{cell}} \;=\; \mathsf{H}\big(H_1 \parallel H_2 \parallel \;\ldots\; \parallel H_G\big).$$

Definition 6.2 (Write-Ahead Log Frame)**.** A cache frame journaling the memoized DAG nodes is serialized as a length-prefixed frame:

$$\text{WAL\_Frame} \;=\; \langle\, \text{length},\; \mathsf{H}(\text{payload}),\; \text{payload} \,\rangle, \quad (4)$$

allowing recovery to discard torn frames.

7 The Refusal Register

Refusal 7.1 (No path dependencies on knhk). Verified by live probes, recorded as the standing reason: the leanest candidate crate drags tokio/reqwest/opentelemetry transitively; the μ-kernel ships property-testing libraries as regular dependencies and uses unsafe transmutes; the workspace does not build as a whole. Ports of 50–200-line designs, tagged `PORT(knhk)` with per-item deltas, cost less than the supply chain. Drift is greppable.

Refusal 7.2 (No measured CPU ticks). knhk's own hot receipts carried dummy tick values behind a hardcoded 4 GHz assumption. Declared deterministic costs (Ticks as an abstract unit) are the honest model for a planner; rdtsc instrumentation is a bench follow-up, documented as a divergence, not smuggled in as a claim.

Refusal 7.3 (No novelty-curve-under-faults measurement — yet). Member-level fault injection recovers without re-running solver work, so a work-proxy re-measurement would *overstate* the cache dividend — the flattering error. The claim is withheld until node-level injection is wired through the fleet path; the receipt file names this in its own deferred list.

Refusal 7.4 (v1 scope: beat scheduling, full R1/W1/C1 SLO matrix, quarantine parole, supra-cell supervision). Each receipted with salvage: the enum and one tier transition ship; epoch boundaries already provide the re-admission cadence beats would refine; quarantined classes await a parole mechanism; cells supervising cells is the composition step after this one.

8 Conclusion: What Derivation Now Covers

Edition one: the *plan* is derived — order and bindings discovered, not authored. Edition two: every derivation stands on a named supply chain. This edition: the *crash space* is derived — the supervision tree earned from the plan's own dependency structure, the failure branches mined from the solver's own fragility analysis, both hashed into the same receipt lineage as the plan, so that at runtime every failure either lands in a branch that existed before the failure did, or surfaces as an honest, unshadowable gap.

Armstrong made failure survivable when the crash space was unknowable. The instruments made hidden events reconstructable by predicting their signatures. The lineage made execution bounded and sealed. The synthesis substrate adds the step none of them had: *the machine that derives the work now derives the failure of the work* — and measures, rather than asserts, that supervision costs nothing on the healthy path, tracks novelty rather than rate under faults, and converts recurring pathology into standing refusals by quorum.

Let it crash was the right answer when the crash space was unknowable.
Derive the crash space is the answer when the planner can see it first.

A trillion agents do not need a trillion failure handlers.
They need one law: every crash lands in a named branch, or names a gap — and either way, there is a receipt.

References

[1] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors.* PhD thesis, KTH, 2003. (Supervision trees, restart intensity, "let it crash.")

[2] The ATLAS Collaboration. *The ATLAS Trigger System.* JINST 15, 2020. (Predicted event signatures as the admission gate for measurement.)

[3] Event Horizon Telescope Collaboration. *First M87 Event Horizon Telescope Results I.* ApJL 875, 2019. (Reconstruction from lawful projections of an instrument designed around a predicted signal.)

[4] S. Chatman. *CNS/BitActor: the 8T/8H/8M doctrine and the canonical tick-bounded engine.* In-house, 2025. `cns/bitactor` (`BITACTOR_TICK_BUDGET 8`, zero-tick fast path, BLAKE3 spec↔exec sealing).

[5] S. Chatman. *ByteActor V3.* In-house, 2025. 64-byte crystal envelopes, bounded SPSC rings, two-realm architecture; honest V2 validation recorded its own 8-tick FAIL.

[6] S. Chatman. *knhk/Genesis: Knowledge-graph hot-path engine.* In-house, 2026. `genesis-mu-kernel` (TickBudget, ChatmanBounded, signed ReceiptChain), `genesis-etl` (RuntimeClass R1/W1/C1, ParkManager, failure actions) — the classification plane this edition's actuation layer completes.

[7] S. Chatman. *The Synthesis Prototype* (first edition) and *. . . With Its Supply Chain* (second edition). Praxis, `docs/thesis/`, Genesis Day 9 (2026). Derived plans; the measured overlap curve; the scientific lineage of each layer.

[8] Ericsson AB. *OTP Design Principles: Supervision Trees.* Erlang/OTP documentation. (one_for_one / rest_for_one / one_for_all; max restart intensity.)

[9] S. Chatman. *Beyond Cognitive Load, Within Admissible Projection.* Praxis, `docs/thesis/projection_thesis.pdf`, Genesis Day 1 (2026).